

ComicVerseAI: An AI-Powered Comic Story and Artwork Generator

Project Documentation & Implementation Report

Team Leader: P.V.Poornima

Team Members:

- 1. Prathistha S
- 2. Sandhiya R
- 3. Kiruthika.K

Name	Role	Main Project Tasks	Video & Media Tasks
P.V. Poornima	Team Leader	Backend development (FastAPI) & Gemini AI integration	Voiceover narration
Prathistha S	Team Member	Frontend UI design & API endpoint testing	Video editing & transitions
Sandhiya R	Team Member	Image generation pipeline & file storage setup	Video editing & audio syncing
Kiruthika.K	Team Member	Layout assembly, static assets & documentation	Report writing & screenshots

1. Introduction & System Overview

1.1 Abstract

ComicVerseAI is an automated web application designed to generate complete comic strips from simple user text prompts. The application leverages Google Gemini AI models for story script creation, character prompt structuring, and scene narration, paired with an AI image generation pipeline to render individual comic panel artwork. Built on top of a FastAPI backend and a clean HTML/CSS web interface, ComicVerseAI streamlines the creative process of converting conceptual story ideas into visual sequential art.

1.2 Key System Features

- **Automated Script & Prompt Generation:**
Uses Gemini models (`gemini_flash.py` / `gemini_pro.py`) to break down user prompts into multi-panel comic scripts with scene descriptions and narrations.
- **AI Image Generation:** Dynamically generates panel artwork tailored to specified comic art styles via `image_generator.py`.

- **Modular Panel & Layout Processing:**
Organizes individual generated panel assets within `static/panels/` and builds structured comic layouts using `layout_builder.py` and `exporters.py`.
- **Interactive Web Interface:** Provides an intuitive web interface (`templates/index.html`, `templates/comic_preview.html`, and `templates/export_success.html`) for user input, real-time previewing, and downloading finalized comics.

2. Project Architecture & Directory Structure

2.1 Directory Layout

The project follows a modular FastAPI directory layout:

- **app/**: Contains core Python application logic, API route endpoints, Gemini integration modules, image generator handlers, and layout assembly scripts.
- **static/**: Houses static web assets including rendered panel images (`static/panels/`), exported comic files (`static/exports/`), and typography assets (`static/fonts/`).

- **templates/**: Stores Jinja2 HTML templates for the user interface screens.
- **comiccraft-env/**: Python virtual environment storing project dependencies.

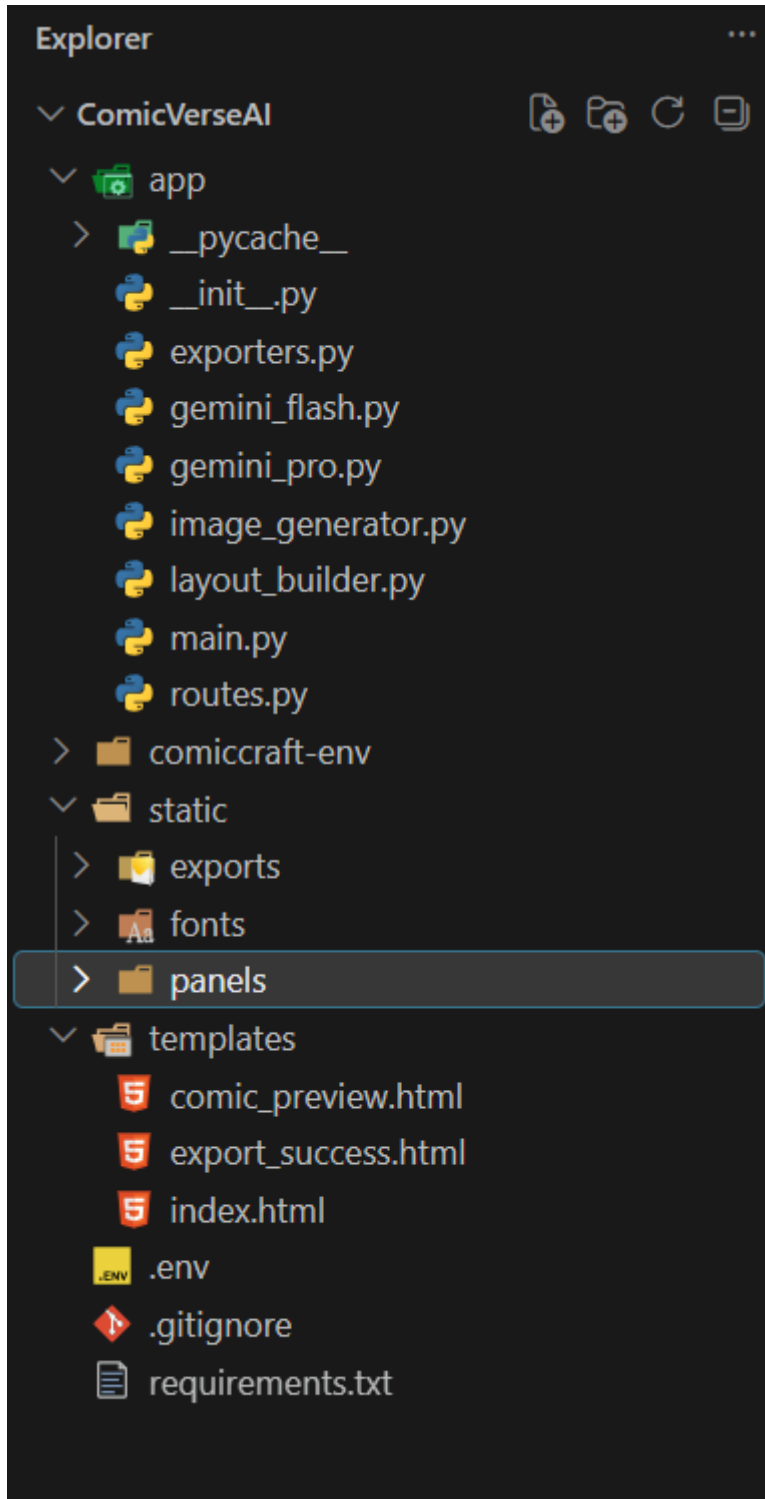
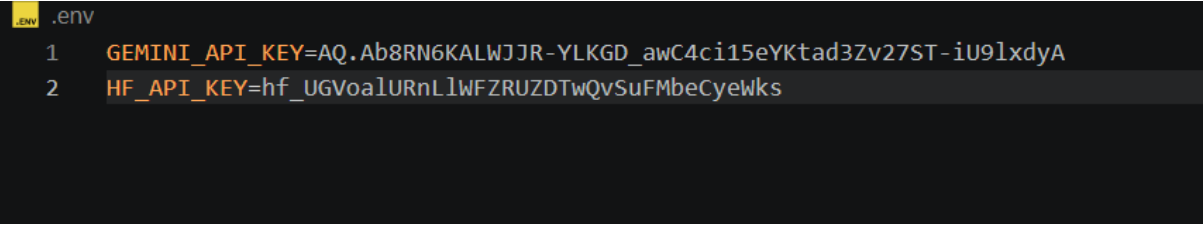


Figure 1: ComicVerseAI Directory and File Structure in VS Code Explorer.

3. Core Implementation & Code Highlights

3.1 Environment Configuration (.env)

Environment variables store sensitive access keys and default generation parameters, including the API key for Google Gemini and storage path definitions.



```
.env
1 GEMINI_API_KEY=AQ.Ab8RN6KALWJJR-YLKGD_awC4ci15eYKtad3Zv27ST-iU9lxdyA
2 HF_API_KEY=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
```

Figure 2: Environment Configuration (.env File).

3.2 FastAPI Application Entry Point & Routes (app/main.py & app/routes.py)

The application backend is built using FastAPI.

`main.py` initializes the web application, mounts static file directories, and configures Jinja2 template rendering. `routes.py` handles user submissions, calls generation pipelines, and serves HTML templates.

```

app > main.py
1  import os
2  from dotenv import load_dotenv
3
4  # Load .env file before importing routes
5  load_dotenv()
6
7  from fastapi import FastAPI
8  from fastapi.staticfiles import StaticFiles
9  from app.routes import router
10
11  app = FastAPI(title="ComicCraft AI")
12
13  # Ensure required directories exist
14  os.makedirs("static/panels", exist_ok=True)
15  os.makedirs("static/exports", exist_ok=True)
16  os.makedirs("static/fonts", exist_ok=True)
17
18  app.mount("/static", StaticFiles(directory="static"), name="static")
19  app.include_router(router)

```

This is [main.py](#)

```

1  import traceback
2  import os
3  from fastapi import APIRouter, Request, Form, HTTPException
4  from fastapi.responses import HTMLResponse, FileResponse
5  from fastapi.templating import Jinja2Templates
6  from pydantic import BaseModel
7
8  from app.gemini_flash import generate_outline
9  from app.gemini_pro import generate_story
10 from app.image_generator import generate_image
11 from app.layout_builder import build_comic_layout
12 from app.exporters import save_pdf
13
14 router = APIRouter()
15 templates = Jinja2Templates(directory="templates")
16
17 class PromptRequest(BaseModel):
18     prompt: str
19     character_name: str
20     setting: str
21     tone: str
22     style: str
23
24 @router.get("/", response_class=HTMLResponse)
25 async def index(request: Request):
26     return templates.TemplateResponse(request=request, name="index.html")

```

Figure 3: FastAPI Application Entry Point and Route Definitions.

3.3 Gemini AI Integration (`app/gemini_flash.py` / `app/gemini_pro.py`)

Gemini AI processes the raw text input entered by the user. It structures the narrative into sequential panels, providing both detailed image prompts for the generator and story narration text for each panel

```
app > gemini_pro.py
1  import os
2  import time
3  from google import genai
4
5  client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
6
7  MODELS_TO_TRY = ["gemini-3.5-flash-lite", "gemini-1.5-flash", "gemini-1.5-pro"]
8
9  def generate_story(outline: list) -> str:
10     """
11     Generates a detailed comic story with narration and character dialogue
12     from a list of comic panel outlines using Gemini.
13
14     Args:
15         outline (list): A list of strings representing each comic panel's idea.
16
17     Returns:
18         str: The generated comic story text or an error message.
19     """
20     # Format the panel outline as a numbered list for clarity
21     formatted_outline = "\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])
22
23     # Construct the prompt
24     prompt = f"""
25     You're a comic book writer.
```

```
27     Given the following panel breakdown, write a comic-style story with engaging narration
28
29     Panel Outline:
30     {formatted_outline}
31
32     Guidelines:
33     - Use a fun and engaging tone, like an actual comic book.
34     - Include narration and clearly marked character lines.
35     - Keep each panel self-contained but part of a cohesive story.
36     """
37
38     for model_name in MODELS_TO_TRY:
39         for attempt in range(3):
40             try:
41                 response = client.models.generate_content(
42                     model=model_name,
43                     contents=prompt,
44                 )
45                 return response.text
46             except Exception as e:
47                 if "503" in str(e) or "404" in str(e):
48                     time.sleep(2 * (attempt + 1))
49                     continue
50                 return f"Error generating story: {str(e)}"
51
52     return "Error generating story: Gemini API endpoints are temporarily unavailable."
```

Figure 4: Gemini Script and Prompt Engineering Integration.

3.4 Image Generation Module (`app/image_generator.py`)

The `image_generator.py` script takes structured prompts from Gemini, triggers the image rendering pipeline, and saves output image files directly into the `static/panels/` directory.

```
pp 7  image_generator.py
1  import os
2  import re
3  import torch
4  from diffusers import StableDiffusionPipeline
5
6  # Suppress Hugging Face symlinks warning on Windows
7  os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
8
9  # Initialize local Stable Diffusion pipeline on CPU
10 model_id = "runwayml/stable-diffusion-v1-5"
11 pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=torch.float32)
12 pipe = pipe.to("cpu")
13
14 def sanitize_filename(prompt: str) -> str:
15     clean = re.sub(r'^\w\s-', '', prompt).strip().lower()
16     clean = re.sub(r'[-\s]+', '_', clean)[:30]
17     return f"{clean}.png"
18
19 def generate_image(prompt, filename=None):
20     if not filename:
21         filename = sanitize_filename(prompt)
22
23     image = pipe(prompt).images[0]
24     path = f"static/panels/{filename}"
25     os.makedirs(os.path.dirname(path), exist_ok=True)
26     image.save(path)
27     return path
```

Figure 5: Panel Image Generator Execution Logic.

4. System Execution & Application Workflow

4.1 Server Launch

The backend server is launched via the Uvicorn ASGI server within the activated virtual environment (`comiccraft-env`)

```
INFO:      Waiting for application startup.  
INFO:      Application startup complete.  
INFO:      127.0.0.1:63918 - "GET / HTTP/1.1" 200 OK  
INFO:      127.0.0.1:63918 - "GET / HTTP/1.1" 200 OK
```

Figure 6: Uvicorn Server Startup in VS Code Terminal.

4.2 Web Interface & Input Processing

Users access the system through [\[http://127.0.0.1:8000\]](http://127.0.0.1:8000)(<http://127.0.0.1:8000>). The input interface ([index.html](#)) lets users submit story ideas and select comic art styles.

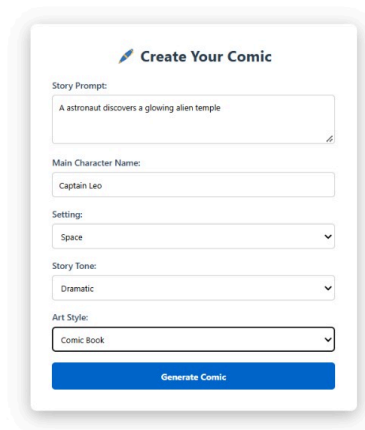
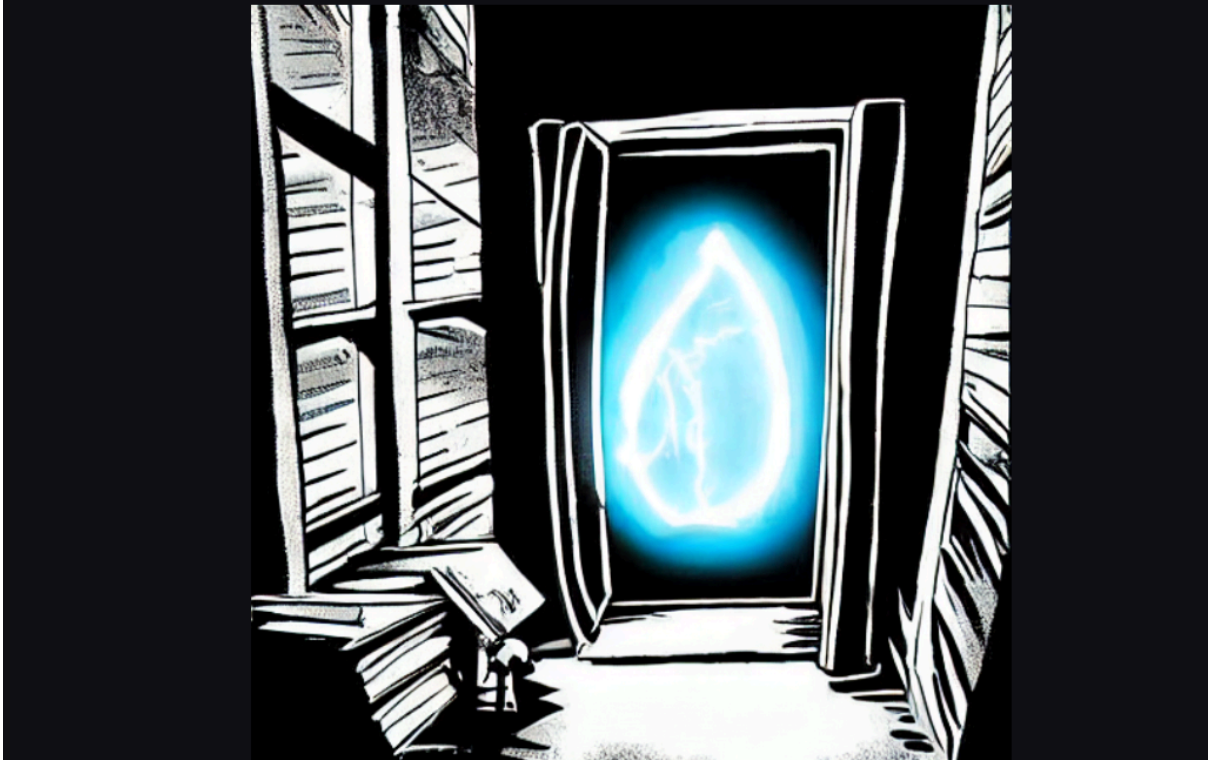
The image shows a web form titled "Create Your Comic" with a blue pencil icon. It contains several input fields: a text area for "Story Prompt" with the example text "A astronaut discovers a glowing alien temple", a text field for "Main Character Name" with "Captain Leo", a dropdown menu for "Settings" with "Space", a dropdown menu for "Story Tone" with "Dramatic", and a dropdown menu for "Art Style" with "Comic Book". A blue button labeled "Generate Comic" is at the bottom.

Figure 7: User Input Interface ([index.html](#)).

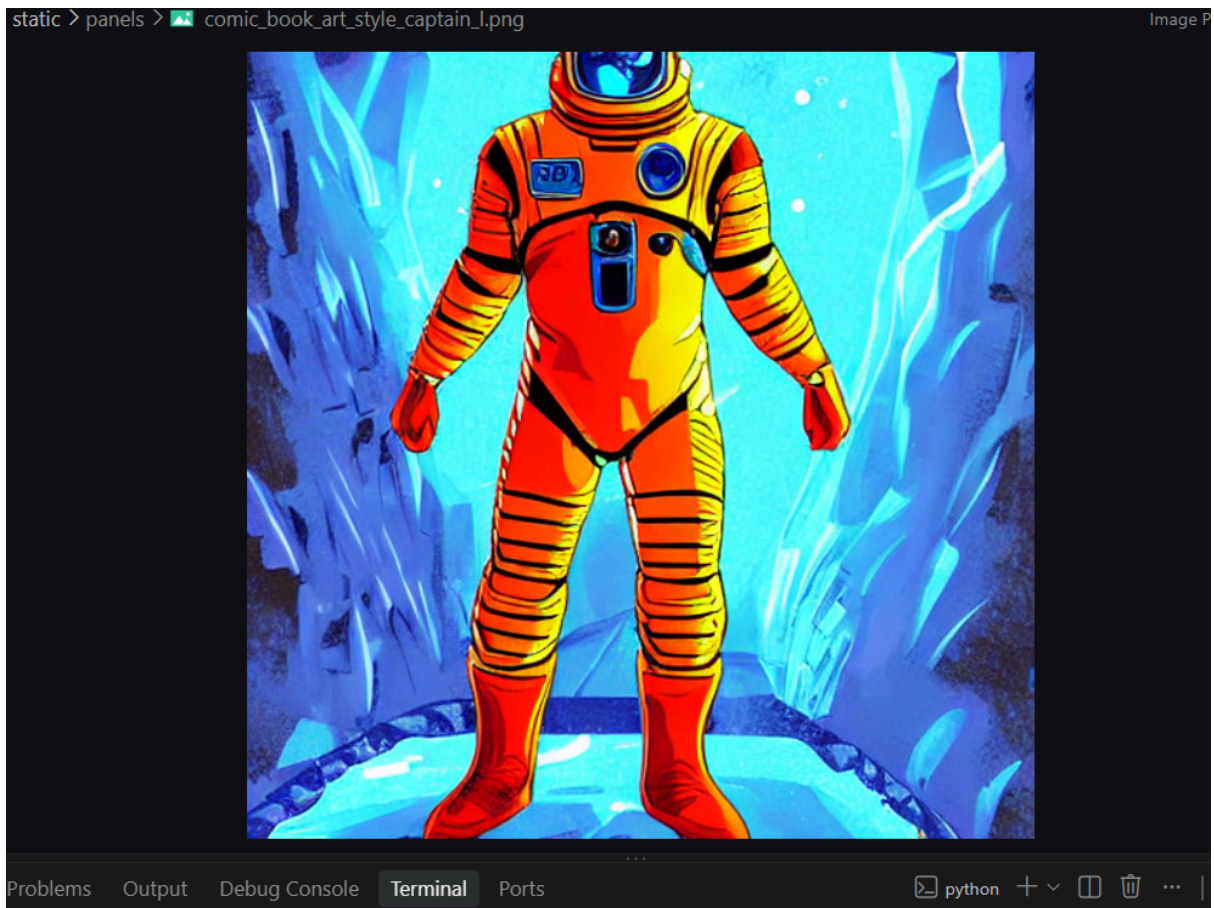
5. Output Demonstration & Story Narration

5.1 Generated Comic Panels & Narrations

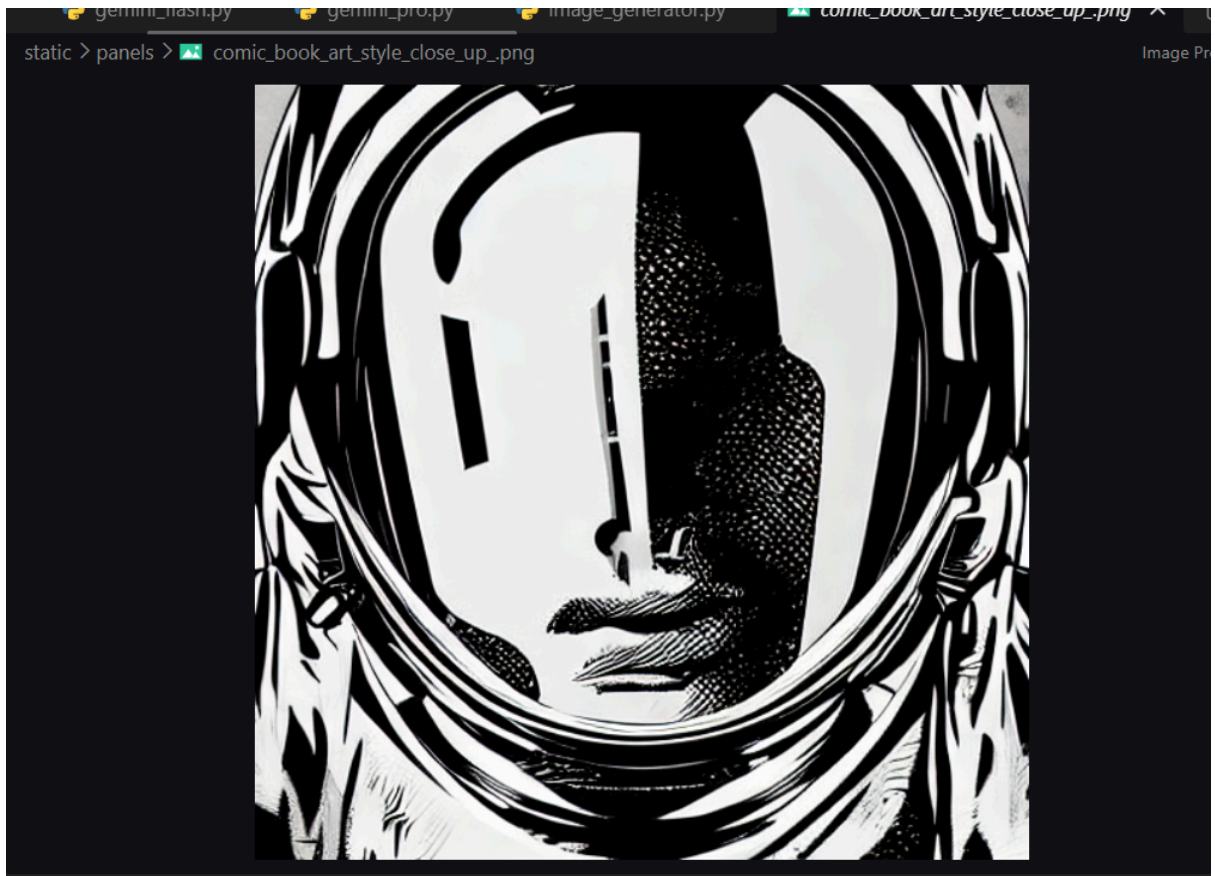
The system generates visual artwork along with structured narrative text created by Gemini.



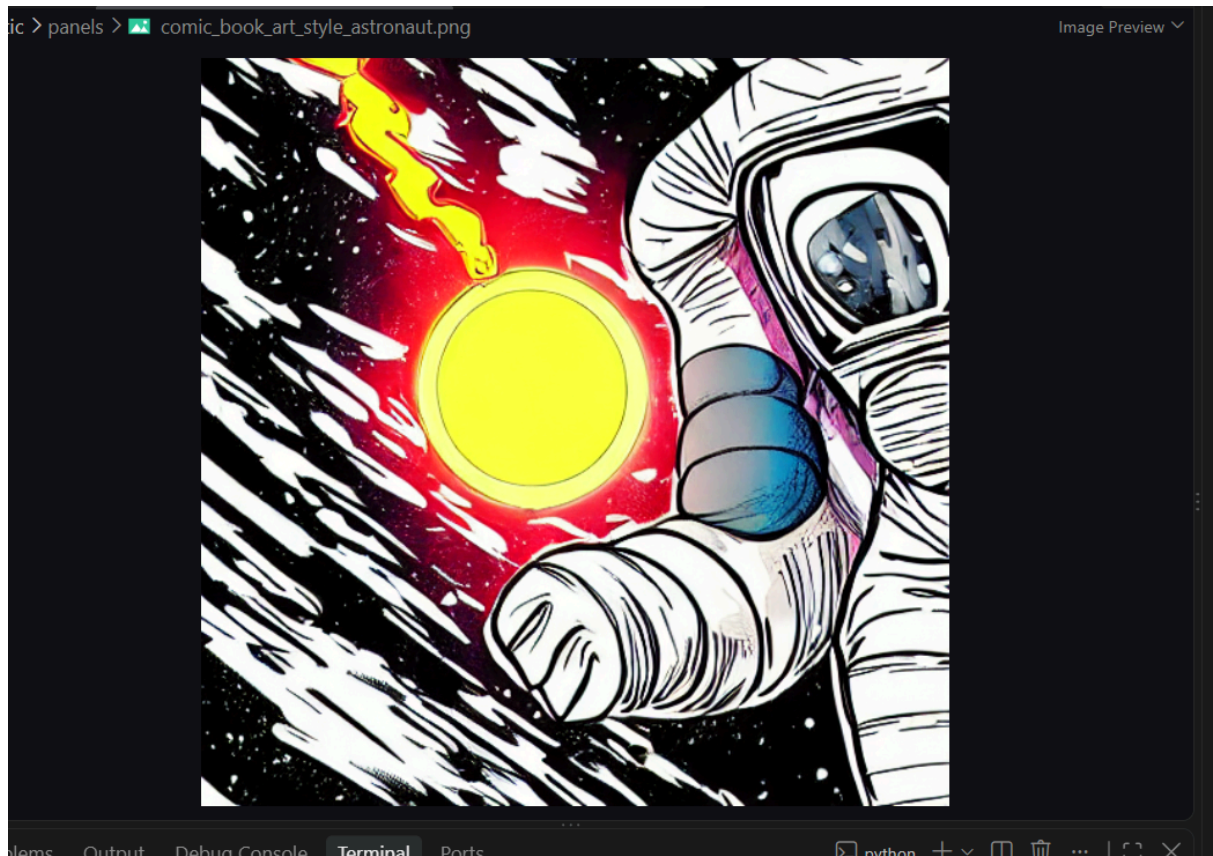
Story Narration: *"An unexpected energy spike destabilizes the station as a brilliant blue rift tearing through space suddenly opens before Captain Leo."*



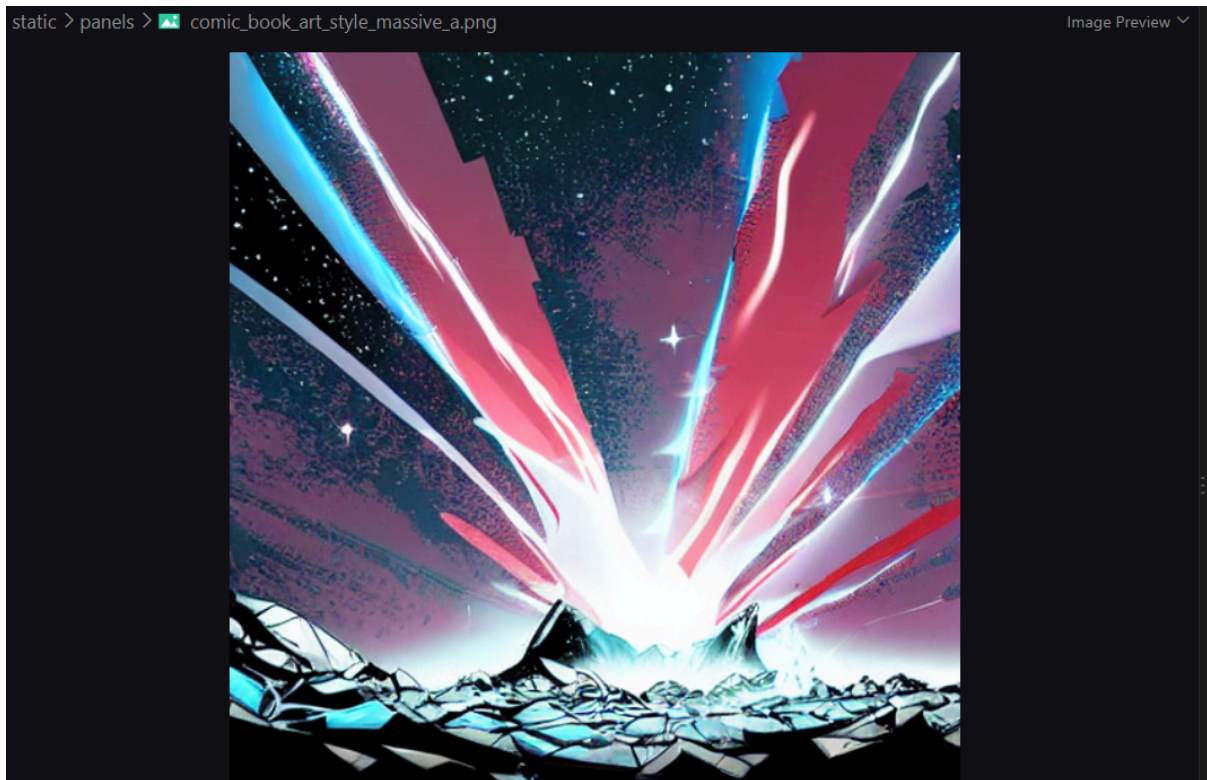
Story Narration: *"Suited up and standing resolute against the unknown radiation, Captain Leo prepares to step into the glowing portal."*



Story Narration: *"With calm focus behind the visor, Leo locks targets and calculates the trajectory through the dimensional rift."*



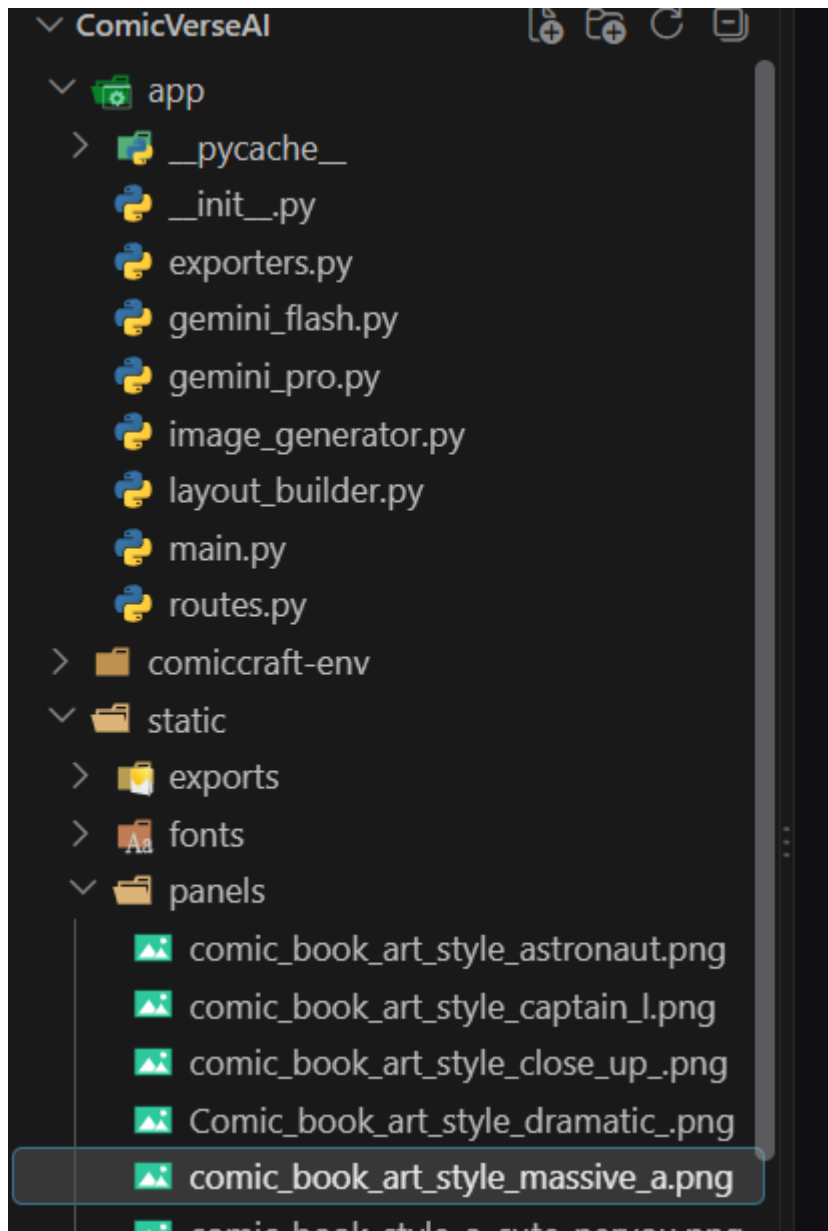
Story Narration: *"Emerging on the other side, Leo floats through deep space toward a mysterious glowing celestial core."*



Story Narration: *"Reaching out to touch the artifact triggers a massive surge of cosmic energy across the galaxy."*

5.2 Storage & File Export

Once the pipeline finishes processing, the output images are organized inside the `static/panels/` directory, ready to be integrated into layouts or exported.



*Figure 8: AI-Generated Comic Panel Outputs Saved in **static/panels/** Directory.*

6. Conclusion & Future Scope

6.1 Conclusion

ComicVerseAI demonstrates an end-to-end implementation combining large language models and diffusion-based image generators. By leveraging FastAPI and modular Python components, the project

successfully automates sequential comic script composition, panel rendering, and web-based distribution.

6.2 Future Scope

- **Direct Speech-Bubble Rendering:** Expanding `layout_builder.py` to automatically composite speech bubbles and typography onto panel images using Pillow and custom fonts from `static/fonts/`.
- **Character Consistency:** Implementing reference image embedding pipelines to maintain identical facial identity and clothing across multi-page comic chapters.
- **PDF Export Improvements:** Enhancing `exporters.py` to allow multi-page high-resolution PDF downloads for printing.

